

小红书AI前端面试

一面（技术基础） · 1-10

1. 实现一个类似 AI 聊天对话框的 CSS 布局，要求左侧头像固定，右侧内容自适应，且内容过长时自动换行。
2. 请说明在处理 AI 流式输出（Streaming）时，Fetch API 与传统的 XMLHttpRequest 有什么区别？
3. 手写代码：实现一个简单的流式数据解析器，将 SSE（Server-Sent Events）格式的字符串转换为对象。
4. 在 TypeScript 中，如何定义一个支持多种 AI 模型配置的通用接口？
5. 说一下 JavaScript 的事件循环（Event Loop）以及它如何影响 AI 字符打字机效果的渲染？
6. 如何实现一个带有自动滚动到底部功能的对话列表，且用户手动向上滚动时停止自动滚动？
7. 解释一下 Promise.all 和 Promise.allSettled 的区别，在并行调用多个 AI 模型时你会选哪个？
8. 简单实现一个防抖函数（Debounce），并说明它在 AI 搜索建议（Query Suggestion）场景下的作用。
9. CSS 中如何实现一个无限旋转的 Loading 动画，用于表示 AI 正在思考？
10. 数组转树结构：AI Agent 的任务拆解通常是树状的，请写一个函数将扁平的任务列表转为嵌套树。

二面（深入原理与项目） · 1-10

1. 详细说明 SSE（Server-Sent Events）的工作原理，以及为什么 AI 对话场景不选择 WebSocket？
2. 在 React 中渲染长达数万字的 AI 回复时，如何保证页面的流畅度？
3. AI 输出的 Markdown 包含代码块、公式（MathJax/KaTeX），前端如何实现实时解析与高亮？
4. 谈谈你在项目中是如何进行 Prompt Engineering 的前端管理策略的？
5. 如何实现 AI 对话中的“中止生成”功能？请从前端请求链路说明。
6. 复杂的 AI Agent 交互中，如何管理多步任务的状态流转？
7. 针对 AI 打字机效果，如何优化以减少浏览器的重排与重绘？
8. 如果 AI 返回的内容非常大，导致前端内存溢出或卡死，你会怎么排查和解决？
9. 如何防范 AI 输出内容带来的安全风险（如 Prompt Injection 或 XSS）？
10. 谈谈 Web Worker 在 AI 前端应用中的具体场景。

三面（架构与综合能力） 1-6

1. 如果让你从零设计一个企业级的 AI Chat 组件库，你会考虑哪些核心模块？

2. 在RAG（检索增强生成）系统中，前端可以做哪些优化来提升用户体验？
3. 如何建立一套 AI 前端性能监控指标？除了LCP、FID 还要关注什么？
4. 面对快速迭代的AI技术（如从 GPT-3.5到GPT-4，再到各种插件机制），前端架构如何保持灵活性？
5. 在团队协作中，你如何利用AI 工具（如 Copilot, Cursor）提升前端开发效率？
6. 聊聊你在过去一年中遇到的最具有挑战性的AI前端问题，你是如何解决的？

一面（技术基础）

1. 实现一个类似 AI 聊天对话框的 CSS 布局，要求左侧头像固定，右侧内容自适应，且内容过长时自动换行。

难度：★

考点：CSS 布局基础

答案要点：

通常使用 Flex 布局或 Grid 布局来实现，通过设置 flex-shrink 为 0 保证头像不被压缩。

- 使用 display: flex 容器，头像部分设置固定宽度
- 内容部分设置 flex: 1 占据剩余空间，并配合 word-break: break-word 处理长文本
- 考虑多轮对话场景，使用 flex-direction: column 管理垂直列表
- 针对 AI 返回的 Pre 代码块，需要设置 overflow-x: auto 防止撑破布局

常见坑：

- 忘记设置 min-width: 0 导致 flex 子元素内容溢出
- 忽略了移动端适配，头像在小屏下占用空间过多

追问：

- 如果内容区包含一个很长的连续英文字符串，你会怎么处理？
- 如何实现对话气泡的小三角指向效果？

2. 请说明在处理 AI 流式输出（Streaming）时，Fetch API 与传统的 XMLHttpRequest 有什么区别？

难度：★★

考点：网络请求与流处理

答案要点：

Fetch API 原生支持 ReadableStream 接口，允许在数据完全到达前逐块处理，而 XHR 通常需要等待全部加载或通过监听 progress 事件手动切片。

- Fetch 的 response.body 是一个 ReadableStream，可以通过 getReader() 异步迭代读取
- XHR 虽然有 readyState 3（LOADING），但对二进制流的支持和 API 易用性远不如 Fetch

- Fetch 配合 async/await 语法更加简洁，天然适合处理 LLM 的 SSE 响应
- 内存占用方面，Fetch 流式处理可以边读边丢弃，避免大文本一次性载入内存

常见坑：

- 以为 Fetch 的 response.json() 也能流式处理（实际上它会等待流结束）
- 忽略了 Fetch 不会自动携带 Cookie 的默认行为

追问：

- 如何在 Fetch 中实现请求超时中断？
- 简单描述下 ReadableStream 的读取逻辑。

3. 手写代码：实现一个简单的流式数据解析器，将 SSE（Server-Sent Events）格式的字符串转换为对象。

难度：★★

考点：字符串处理、正则表达式

答案要点：

SSE 格式通常以 data: 开头，以两个换行符分隔，需要通过字符串分割或正则提取内容并解析 JSON。

- 识别 data: 前缀并截取后续内容
- 处理多行数据合并的情况
- 使用 try...catch 包裹 JSON.parse 以防模型输出非法格式
- 过滤掉 SSE 协议中的 [DONE] 等特殊标识符

代码块

```
1  function parseSSE(chunk) {
2    const lines = chunk.split('\n');
3    const results = [];
4    for (const line of lines) {
5      if (line.startsWith('data: ')) {
6        const data = line.slice(6);
7        if (data === '[DONE]') break;
8        try {
9          results.push(JSON.parse(data));
10       } catch (e) {
11         console.error('Parse error', e);
12       }
13     }
14   }
15   return results;
16 }
```

常见坑：

- 一个 chunk 可能包含半个 JSON 字符串，直接解析会报错
- 忽略了 \n\n 的规范分隔符

追问：

- 如果 chunk 截断了 JSON 导致解析失败，你如何缓存残余字符串？

4. 在 TypeScript 中，如何定义一个支持多种 AI 模型配置的通用接口？

难度：★

考点：TS 泛型与联合类型

答案要点：

利用泛型和辨析联合类型（Discriminated Unions）来定义不同模型的参数结构，确保类型安全。

- 定义基础配置接口，包含 model、temperature 等通用字段
- 使用 type 或 interface 结合联合类型区分不同厂商的特定参数
- 利用 keyof 和索引签名增强扩展性
- 应用泛型约束来处理 API 响应的 Payload

常见坑：

- 使用 any 定义模型参数，导致失去类型检查
- 联合类型没有公共的辨识属性（如 type 字段）

追问：

- 什么是 TS 的类型守卫（Type Guard），在处理模型返回时怎么用？

5. 说一下 JavaScript 的事件循环（Event Loop）以及它如何影响 AI 字符打字机效果的渲染？

难度：★★

考点：事件循环、渲染机制

答案要点：

事件循环负责协调同步任务、微任务和宏任务，打字机效果如果处理不当会因为大量微任务阻塞 UI 渲染。

- 每一轮 Event Loop 结束后，浏览器会尝试进行 UI Rendering
- AI 流式返回的数据通常是微任务（Promise），频繁更新状态会导致渲染压力
- 使用 requestAnimationFrame 可以将打字机效果同步到浏览器的刷新频率
- 避免在主线程进行复杂的 Markdown 解析计算，防止掉帧

常见坑：

- 在循环中直接操作 DOM 而不是通过状态驱动

- 认为 `setTimeout(0)` 和 `requestAnimationFrame` 是等价的

追问：

- 如果 AI 返回速度极快（每秒 100 词），你会如何优化渲染频率？

6. 如何实现一个带有自动滚动到底部功能的对话列表，且用户手动向上滚动时停止自动滚动？

难度：★★

考点：DOM API、交互逻辑

答案要点：

通过监听容器的 `scroll` 事件判断用户是否处于底部，并在数据更新时根据状态决定是否调用 `scrollIntoView`。

- 记录上一次的 `scrollTop` 和 `scrollHeight`
- 判断公式： $\text{scrollHeight} - \text{scrollTop} - \text{clientHeight} < \text{阈值}$ （如 50px）
- 使用 `ResizeObserver` 监听内容高度变化
- 用户手动向上滚动时修改 `isAtBottom` 标志位

常见坑：

- 忽略了图片或公式异步加载导致的高度变化
- 滚动动画执行过程中再次触发滚动导致的抖动

追问：

- 这种场景下使用 `scrollIntoView({ behavior: 'smooth' })` 有什么潜在问题？

7. 解释一下 `Promise.all` 和 `Promise.allSettled` 的区别，在并行调用多个 AI 模型时你会选哪个？

难度：★

考点：异步并发控制

答案要点：

`Promise.all` 在任意一个请求失败时就会立即 `reject`，而 `Promise.allSettled` 会等待所有请求完成并返回每个请求的状态。

- 并行调用多个模型对比结果时，通常选 `allSettled`，因为不能因为一个模型超时导致所有结果不可见
- `Promise.all` 适用于强依赖场景，`allSettled` 适用于结果收集场景
- `allSettled` 返回的是对象数组，包含 `status`、`value` 或 `reason`
- 性能上两者基本一致，主要是逻辑处理上的差异

常见坑：

- 忘记处理 `Promise.all` 失败时的 `catch` 逻辑

- 在 allSettled 后忘记过滤出 fulfilled 的结果

追问：

- 如果要实现“最快返回的模型结果”，应该用哪个 API？

8. 简单实现一个防抖函数（Debounce），并说明它在 AI 搜索建议（Query Suggestion）场景下的作用。

难度：★

考点：高阶函数、性能优化

答案要点：

防抖函数通过闭包维护一个定时器，在规定时间内多次触发只执行最后一次，减少无效的 API 请求。

- 核心逻辑：clearTimeout + setTimeout
- 作用：避免用户每输入一个字符就调用一次昂贵的 LLM 接口
- 提升用户体验，减少服务端压力和 Token 消耗
- 需支持配置延迟时间和立即执行选项

代码块

```
1  function debounce(fn, delay) {
2    let timer = null;
3    return function(...args) {
4      if (timer) clearTimeout(timer);
5      timer = setTimeout(() => fn.apply(this, args), delay);
6    }
7  }
```

常见坑：

- 闭包内的 this 指向问题
- 忘记在组件销毁时清除定时器

追问：

- 防抖和节流（Throttle）在 AI 交互中分别适用什么场景？

9. CSS 中如何实现一个无限旋转的 Loading 动画，用于表示 AI 正在思考？

难度：★

考点：CSS 动画

答案要点：

使用 @keyframes 定义旋转路径，并通过 animation 属性应用到元素上。

- 定义 0% 到 100% 的 transform: rotate(0deg) 到 rotate(360deg)
- 设置 animation-iteration-count 为 infinite

- 使用 linear 速度曲线保证旋转平滑
- 考虑使用 transform: translateZ(0) 开启硬件加速

常见坑：

- 动画导致页面重排（Reflow），应尽量使用 transform 而不是 top/left
- 忘记设置 transform-origin 导致旋转中心偏移

追问：

- 如何用 CSS 实现一个类似 ChatGPT 的闪烁光标效果？

10. 数组转树结构：AI Agent 的任务拆解通常是树状的，请写一个函数将扁平的任务列表转为嵌套树。

难度：★★

考点：数据结构转换

答案要点：

利用 Map 存储所有节点引用，通过遍历一次数组，根据 parentId 将子节点归类到父节点的 children 属性中。

- 初始化一个 Map，以 id 为 key 存储每个节点
- 再次遍历，若有 parentId 则将其加入父节点的 children
- 若无 parentId，则视为根节点
- 时间复杂度要求 $O(n)$

代码块

```
1  function arrayToTree(items) {
2      const result = [];
3      const itemMap = {};
4      for (const item of items) {
5          itemMap[item.id] = { ...item, children: [] };
6      }
7      for (const item of items) {
8          const id = item.id;
9          const parentId = item.parentId;
10         const treeItem = itemMap[id];
11         if (parentId === 0 || !parentId) {
12             result.push(treeItem);
13         } else {
14             itemMap[parentId].children.push(treeItem);
15         }
16     }
17     return result;
18 }
```

常见坑：

- 没考虑父节点在数组中出现在子节点之后的情况（Map 预处理可解决）
- 引用传递导致的原始数据污染

追问：

- 如果树的深度非常大，递归处理会有什么风险？
-

二面（深入原理与项目）

1. 详细说明 SSE（Server-Sent Events）的工作原理，以及为什么 AI 对话场景不选择 WebSocket？

难度：★★

考点：通信协议对比

答案要点：

SSE 是基于 HTTP 协议的单向流式传输，相比 WebSocket 更加轻量且天然支持断线重连。

- 原理：服务端设置 Content-Type: text/event-stream，保持长连接持续发送数据
- 优势：AI 场景主要是服务端向客户端单向输出，SSE 协议开销小，兼容性好
- 自动重连：浏览器内置了重连机制，通过 Last-Event-ID 恢复进度
- 穿透性：SSE 是标准 HTTP，更容易穿透防火墙和代理服务器

常见坑：

- 忽略了 HTTP/1.1 下浏览器对同一域名有 6 个连接限制的问题（需用 HTTP/2）
- 忘记在服务端关闭连接，导致资源泄漏

追问：

- 如果需要客户端在中途干预 AI 输出（如停止生成），SSE 如何实现？

2. 在 React 中渲染长达数万字的 AI 回复时，如何保证页面的流畅度？

难度：★★★

考点：渲染性能优化

答案要点：

核心思路是减少 DOM 节点数和降低渲染频率，通过虚拟列表和任务分片来优化。

- 虚拟列表：只渲染可视区域内的 Markdown 片段，减少 DOM 节点
- 状态分片：不要每收到一个 token 就 setState，可以累积一定字符或按时间间隔更新
- memo 与 useMemo：避免对话列表中历史消息的不必要重渲染
- Web Worker：将复杂的 Markdown 转 HTML 逻辑移出主线程

常见坑：

- 频繁触发 `setState` 导致 React 调度压力过大
- 虚拟列表在高度动态变化（AI 正在输入）时的抖动问题

追问：

- 如何计算动态高度的虚拟列表项高度？

3. AI 输出的 Markdown 包含代码块、公式（MathJax/KaTeX），前端如何实现实时解析与高亮？

难度：★★

考点：第三方库集成与性能

答案要点：

使用 `markdown-it` 或 `marked` 作为解析器，配合 `highlight.js` 或 `prismjs` 实现代码高亮，公式则需集成 `KaTeX`。

- 增量解析：只解析新到达的文本片段（较难实现，通常是全量解析但做节流）
- 安全过滤：使用 `DOMPurify` 防止 AI 输出恶意脚本（XSS）
- 异步加载：代码高亮插件较大，应按需动态导入语言包
- 样式隔离：确保 Markdown 样式不会污染全局 CSS

常见坑：

- 实时解析公式时，`KaTeX` 的渲染开销可能导致打字机效果卡顿
- 忘记处理未闭合的 Markdown 标签（如代码块的 `` ```）

追问：

- 如果 AI 输出一半时断开了，如何补全未闭合的代码块标签？

4. 谈谈你在项目中是如何进行 Prompt Engineering 的前端管理策略的？

难度：★★

考点：工程化实践

答案要点：

前端主要负责 Prompt 模板的构建、变量替换以及多轮对话上下文的剪裁管理。

- 模板化：将 System Prompt 和 User Prompt 抽离为配置文件，支持变量插值
- 上下文管理：根据 Token 限制（Window Size）动态截断历史消息，保留最重要的信息
- 结构化输出：通过 Prompt 强制模型返回 JSON，前端使用 JSON Schema 校验
- 版本控制：对不同的 Prompt 版本进行 A/B 测试，记录用户反馈

常见坑：

- 直接在代码里硬编码长字符串 Prompt，难以维护
- 忽略了不同模型对 Prompt 格式（如 ChatML）的要求差异

追问：

- 如何在前端计算 Token 数量以防止超出模型限制？

5. 如何实现 AI 对话中的“中止生成”功能？请从前端请求链路说明。

难度：★★

考点：异步控制

答案要点：

使用浏览器原生的 AbortController 来取消 Fetch 请求，并通知后端停止模型推理。

- 创建 `controller = new AbortController()`，将 signal 传给 fetch
- 用户点击停止时调用 `controller.abort()`
- 前端捕获 `AbortError`，停止打字机动画并清理状态
- 后端需监听连接断开信号，及时终止昂贵的 GPU 推理任务

常见坑：

- 仅仅前端停止渲染，后端仍在持续计费生成
- 再次发起请求时忘记重置 AbortController 实例

追问：

- 如果是 WebSocket 协议，如何实现类似的中止逻辑？

6. 复杂的 AI Agent 交互中，如何管理多步任务的状态流转？

难度：★★★

考点：状态管理、复杂交互

答案要点：

通常采用有限状态机（FSM）或类似 Redux/XState 的方案来管理 Agent 的思考、调用工具、执行、反馈等状态。

- 定义明确的状态集合：`idle`, `thinking`, `calling_tool`, `executing`, `finished`, `error`
- 使用状态机管理状态转移，防止非法跳转（如从 `idle` 直接到 `finished`）
- 记录每一步的 Trace ID，方便链路追踪和调试
- 实时更新 UI 进度条或步骤条，缓解用户焦虑

常见坑：

- 状态过于分散（`useState` 遍地都是），导致逻辑难以追踪
- 忽略了多步任务中某一步失败后的回滚或重试机制

追问：

- 聊聊 XState 在这种复杂场景下的优势。

7. 针对 AI 打字机效果，如何优化以减少浏览器的重排与重绘？

难度：★★★

考点：浏览器性能

答案要点：

打字机效果本质是频繁的 DOM 更新，优化核心在于减少对布局树的影响。

- 使用 `contain: layout` 属性告知浏览器该元素内容变化不影响外部布局
- 预留高度空间（Min-height），避免内容增加时频繁撑开父容器
- 批量更新：将多个字符的更新合并到一次 `requestAnimationFrame` 中
- 避免在更新期间读取 `offsetTop` 等引起强制同步布局的属性

常见坑：

- 每一行文字增加都触发整个对话列表的重排
- 在打字过程中执行复杂的 CSS 动画

追问：

- `content-visibility` 属性在这里能起到什么作用？

8. 如果 AI 返回的内容非常大，导致前端内存溢出或卡死，你会怎么排查和解决？

难度：★★★

考点：内存管理、性能诊断

答案要点：

通过 Chrome DevTools 的 Memory 面板分析堆快照，定位大字符串或未销毁的监听器。

- 现象排查：观察 Task Manager 中的内存占用曲线，确认是否存在泄漏
- 解决方案：采用流式解析，边解析边渲染，不保留完整的原始大字符串
- 垃圾回收：确保断开连接后及时释放 `ReadableStream` 的 reader
- 离线存储：将历史长对话持久化到 IndexedDB，而不是全部存在内存状态机中

常见坑：

- 全局变量引用了大量的对话历史记录
- 闭包导致旧的请求回调无法被回收

追问：

- 什么是 WeakMap，在 AI 场景下有什么应用？

9. 如何防范 AI 输出内容带来的安全风险（如 Prompt Injection 或 XSS）？

难度：★★

考点：前端安全

答案要点：

AI 输出具有不可预测性，必须将其视为不可信输入进行严格过滤。

- XSS 防御：使用 `DOMPurify` 对 Markdown 转换后的 HTML 进行清洗
- 内容安全策略（CSP）：限制脚本执行和外部资源加载

- 敏感信息脱敏：在发送 Prompt 前，前端正则匹配并拦截手机号、密钥等隐私
- 预防注入：对用户输入进行转义，避免用户通过输入指令操控系统级 Prompt

常见坑：

- 认为模型自带安全过滤，前端就直接使用 dangerouslySetInnerHTML
- 忽略了 AI 可能生成的恶意链接（钓鱼风险）

追问：

- 如果 AI 生成了一个